



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

PROJECT 1 (SUPERVISED LEARNING)

Course : SECP2753 DATA MINING

Semester/Session : 2-2024/2025

Section : 02

Lecturer : Dr Rozilawati binti Dollah @ Md Zain

No.	Group Members	Matric No.
1.	Muhammad Adam bin Razali	A23CS0116
2.	Iman Abadi bin Mohd Nizwan	A23CS0084
3.	Muhammad Afiq Danish bin Mohd Hazni	A23CS0118

1.0 INTRODUCTION

In the current era of data-driven world, educational institutes begin to slowly implement data mining techniques to better understand student performance and its factors. The purpose of this approach is to improve student's learning outcomes as well as their performance in school. The project explores various factors that can influence a student's academic success such as hours studied, parental involvement, attendance rate and many other factors. By analyzing the dataset related to student performance in school, the goal is to uncover the meaningful patterns and insights that might correlate to a student's academic performance and help in the decision making process in education.

The main objective of this project is to apply different classification algorithms to predict student outcomes, specifically their exam scores, motivation level and access to resources. Through this analysis, we aim to test the validity of specific business insights that will be explained further in this report. The dataset used in this project contains various features that can be divided into academic, behavioural and background data. Before building the model, the data was prepared and went through several preprocessing steps to provide clean and meaningful data for the classification algorithms' model.

In order to achieve these goals, multiple classification algorithms were implemented and compared, including K-Nearest Neighbours(KNN), Support Vector Machine(SVM) and Deep Learning. Thus, this project determines which model is best suited and most effective for predicting the student's performance by evaluating each algorithm's performance using accuracy and other metrics.

2.0 BUSINESS INSIGHTS

During the initial phase of our project, we identified several potential business insights based on common assumptions and our poor understanding of the dataset. Therefore, as we delved deeper into the dataset and its features, we decided to refine our business insights to better align with the data's characteristics and the specific predictive capabilities of our chosen classification algorithms. The revised business insights are as presented in the table below:

No .	Insight	Explanation
1.	To predict the measurement of student performance using Exam Score.	This insight aims to develop a predictive model where a student's exam score is used as the primary variable to estimate students' academic performance.
2.	To predict whether motivation level leads to better student performance	This insight focuses on correlation between student performance and their motivation levels.
3.	To predict whether availability of educational resources affect student performance.	This insight seeks to establish if there's a link between the accessibility and utilization of educational resources and a student's academic performance.

3.0 DATA PREPROCESSING AND SAMPLING

For the particular dataset, we went through several preprocessing tasks in order to prepare it for training, testing and evaluation. To make sure our data is consistent and accurate, we plan to clean the data to ensure accuracy and reliability, and transform it into a suitable format.

For data mining tasks to possibly reduce computation time and make patterns more visible to the model, normalize the value to make sure the gap is consistent through data as well as presenting meaningful and actionable insights.

3.1 Data Cleaning

```
## Data Cleaning

# Eliminate missing values
print(dataset.isnull().sum())
dataset = dataset.dropna()

# Eliminate duplicate values
print(dataset.duplicated().sum()) # no duplicated values

# Eliminate inconsistent values
dataset.drop(dataset[dataset['Exam_Score'] > 100].index , inplace = True)
dataset.drop(dataset[dataset['Exam_Score'] < 0].index , inplace = True)
dataset.drop(dataset[dataset['Attendance'] > 100].index , inplace = True)
dataset.drop(dataset[dataset['Attendance'] < 0].index , inplace = True)
dataset.drop(dataset[dataset['Previous_Scores'] > 100].index , inplace = True)
dataset.drop(dataset[dataset['Previous_Scores'] < 0].index , inplace = True)
```

Figure 3.1 Data Cleaning Process

Referring to the figure above, The initial phase of the preprocessing is cleaning the data. It involved several critical steps to ensure reliability of the dataset. Firstly, missing values were addressed by removing any rows containing null entries using `dataset.dropna()`. This ensures only complete records to be evaluated, next, the duplicate entries were checked but none were found, therefore ensuring the uniqueness of the data. Finally illogical or inconsistent data were handled by logical constraint. Entries from exam score, attendance, and previous score that exceed values of 100 or negative numbers were identified and eliminated since it is impossible to reach.

3.2 Target Encoding

```
nominal_features = ['Extracurricular_Activities', 'Internet_Access', 'School_Type', 'Learning_Disabilities', 'Gender',]
ordinal_features = ['Parental_Involvement', 'Access_to_Resources', 'Motivation_Level', 'Family_Income', 'Teacher_Quality', 'Peer_Influence',
'Parental_Education_Level', 'Distance_from_Home']
continuous_features = ['Hours_Studied', 'Attendance', 'Previous_Scores', 'Sleep_Hours', 'Tutoring_Sessions', 'Physical_Activity']

# encoding nominal and ordinal data
for feature in nominal_features:
    ne = LabelEncoder()
    dataset[feature + '_encoded_n'] = ne.fit_transform(dataset[feature])

for feature in ordinal_features:
    oe = OrdinalEncoder()
    dataset[feature + '_encoded_o'] = oe.fit_transform(dataset[[feature]])

# binning continuous data
dataset['Exam_Score_binned'] = dataset['Exam_Score'].apply(categorize_score)
dataset['Hours_Studied_binned'] = dataset['Hours_Studied'].apply(categorize_study_hours)
dataset['Attendance_binned'] = dataset['Attendance'].apply(categorize_attendance)
dataset['Previous_Scores_binned'] = dataset['Previous_Scores'].apply(categorize_previous_score)
dataset['Tutoring_Sessions_binned'] = dataset['Tutoring_Sessions'].apply(categorize_tutoring_sessions)
dataset['Physical_Activity_binned'] = dataset['Physical_Activity'].apply(categorize_physical_activity)
dataset['Sleep_Hours_binned'] = dataset['Sleep_Hours'].apply(categorize_sleep_hours)
binned_continuous_features = ['Hours_Studied_binned', 'Attendance_binned', 'Previous_Scores_binned', 'Sleep_Hours_binned', 'Tutoring_Sessions_binned',
'Physical_Activity_binned', 'Exam_Score_binned']
```

Figure 3.2 Target Encoding Process

Based on the figure above, In this step, we involved both encoding of categorical features as well as binning the continuous variables. For categorical features we first identify it and segregate it into nominal and ordinal features based on whether the feature value has inherent order to it or not. Nominal features were encoded using LabelEncoder class, which assigns a unique numerical number to each category. Ordinal features on the other side, which contained inherent order were encoded using OrdinalClass that preserves the hierarchical relationships for among the values.

For continuous features, all of them were transformed through binning which applied a custom categorization function to group continuous numerical values into discrete intervals. For example, exam scores were divided into 5 categories each ranging by 10. This binning simplifies the data but still maintains the relationships and patterns of the data clearly.

3.2 Data Sampling

```
## Sampling
features_column = [f + '_encoded_n' for f in nominal_features] + [f + '_encoded_o' for f in
ordinal_features if f != 'Access_to_Resources' ] + binned_continuous_features

label = ['Exam_Score_binned', 'Motivation_Level_encoded_o', 'Teacher_Quality_encoded_o',
'Access_to_Resources_encoded_o']

# Creating X and y
X = processed_dataset[features_column].values
y = processed_dataset[label[3]].values

# Creating training and testing set

X_train, X_test, y_train, y_test = train_test_split(X ,y, random_state = 42, train_size = 0.7, test_size =
0.3, stratify = y)
```

Figure 3.3 Data Sampling Process

In this sampling step, we partitioned the dataset into training and testing sets through stratified sampling of 80% training set and 20% testing set by using `train_test_split` function. The 19 input features(X) and 1 target variable(y) are divided to create `X_train`, `y_train`, `X_test`, and `y_test` splits. We also implement a random state to ensure the random split is consistent across multiple devices. Stratification technique was also used to make sure equal proportion of each class of the target variables `y` is created.

3.4 Data Normalization

```
## Normalize the numerical features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.fit_transform(X_test)
```

Figure 3.4 Data Normalization Process

Following the dataset split, all of the processed data is stored and underwent normalization using `StandardScaler` class for the input features, `X`. This method transforms numerical features so that they contain mean of 0 and standard deviation of 1 to even out the

proportion of scales and ranges. It can avoid disproportionate values that influence the model's learning process.

4.0 DATA MINING TASKS

Three classification algorithms were implemented in this project to analyze the student performance dataset based on the chosen insights. Each algorithm was trained using selected features derived from the binning and encoding process.

The main library that is being used for the algorithms is *scikit-learn* library, which is a Python library for machine learning. There are other libraries as well such as pandas and numpy for data manipulation and numerical operations. The details of the algorithms are described below.

4.1 K-Nearest Neighbours

K-Nearest Neighbors (KNN) is a supervised machine learning technique that is frequently used for classification applications in machine learning. It works by identifying the "k" nearest data points (neighbors) to a given input and making predictions based on the majority class. Because KNN makes no assumptions about the underlying data distribution, it is a non-parametric, instance-based learning technique.

```
# Test different distance metrics
k_values = [3, 5, 7, 9, 11, 15, 19]

# Store results for different configurations
results = {}

# Test different k values with multiple metrics (Euclidean distance)
print("\n1. Testing different k values with multiple metrics (Euclidean distance):")
print("-" * 80)

# Store results for different metrics
k_results = {
    'k': [],
    'accuracy': [],
    'precision': [],
    'recall': [],
    'f1_score': []
}
```

Figure 4.1 Predefined k-values


```

# Test different k values with multiple evaluation metrics
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k, metric='euclidean')
    knn.fit(X_train_scaled, y_train)
    y_pred = knn.predict(X_test_scaled)

    # Calculate multiple metrics
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average='weighted', zero_division=0)
    recall = recall_score(y_test, y_pred, average='weighted', zero_division=0)
    f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)

    # Store results
    k_results['k'].append(k)
    k_results['accuracy'].append(accuracy)
    k_results['precision'].append(precision)
    k_results['recall'].append(recall)
    k_results['f1_score'].append(f1)

print(f"k={k:2d}: Accuracy={accuracy:.4f}, Precision={precision:.4f}, Recall={recall:.4f}, F1={f1:.4f}")

```

Figure 4.2 Create and Train KNN model with different k-values

This KNN model uses k values in the first parameter, *n_neighbors*, which is the number of nearest neighbors the algorithm considers when making predictions. A list of k values (k = [3, 5, 7, 9, 11, 15, 19]) is predefined to iterate through when the model is trained. This is to determine the best k value to use to compare the model with the other two models. The *metric* parameter specifies the distance metric used, which in this case is the Euclidean distance, to determine the distance between the new data point and its nearest neighbors. The *k_results* variable is used to store the k values based on the list defined and their corresponding metric results, which are accuracy, precision, recall, and F1 score.

```

# Use F1-score as the primary metric for selecting best k (you can change this)
best_k_overall = best_k_f1
print(f"\nUsing k={best_k_overall} as best overall (based on F1-score)")

# Final model with best k and best distance metric
final_model = KNeighborsClassifier(n_neighbors=best_k_overall, metric='euclidean')
final_model.fit(X_train_scaled, y_train)
final_predictions = final_model.predict(X_test_scaled)

# Evaluate final model
final_accuracy = accuracy_score(y_test, final_predictions)
print(f"\nFinal Model Evaluation:")
print(f"Accuracy: {final_accuracy:.4f}")
print("Classification Report:")
print(classification_report(y_test, final_predictions, zero_division=0))
print("\nConfusion Matrix:")
cm = confusion_matrix(y_test, final_predictions)
print(cm)
cm_display = metrics.ConfusionMatrixDisplay(confusion_matrix = cm, display_labels = student_data['Exam_Score_binned'].unique())
cm_display.plot(cmap='Blues')
plt.title('Confusion Matrix')
plt.show()

```

4.3 Training and Evaluation of KNN model based on best k-value

This code section is used for the final model evaluation using the best k value, which is based on the F1 score as the primary metric to determine the best k value. For insight 1, it was found that the best k value is 5. The best k values of Insight 2 and Insight 3 are 9 and 11, respectively. A confusion matrix can be generated that evaluates the performance of a model by comparing its predictions against the actual values.

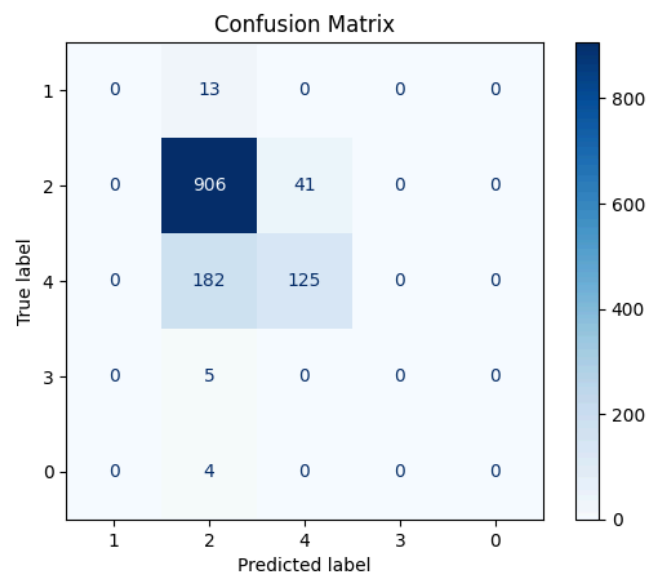


Figure 4.4 Confusion Matrix of KNN model of Insight 1

4.2 Support Vector Machine (SVM)

Support Vector Machine (SVM) is a supervised machine learning algorithm that determines and identifies the optimal decision boundary, called HyperPlane, to separate different classes in high-dimensional space. In this project, the SVM model was trained using the RBF (Radial Basis Function) kernel to allow for non-linear separation of data.

```
features = [col for col in stud_performance.columns if ('_encoded' in col or '_binned' in col) and col != 'Exam_Score_binned']
target = 'Exam_Score_binned'

X = stud_performance[features]
y = stud_performance[target]
```

Figure 4.5 Feature Selection for Insight 1

```
features = [col for col in stud_performance.columns if ('_encoded' in col or '_binned' in col) and col != 'Motivation_Level_encoded_o']
target = 'Motivation_Level_encoded_o'

X = stud_performance[features]
y = stud_performance[target]
```

Figure 4.6 Feature Selection for Insight 2

```
features = [col for col in stud_performance.columns if ('_encoded' in col or '_binned' in col) and col != 'Access_to_Resources_encoded_o']
target = 'Access_to_Resources_encoded_o'

X = stud_performance[features]
y = stud_performance[target]
```

Figure 4.7 Feature Selection for Insight 3

Based on the figures above, the features used for the model were:

- All *_encoded_o/*_encoded_n and *_binned column except the target variable specific for each insight:
 1. Insight 1 - *Exam_Score_binned*
 2. Insight 2 - *Motivation_Level_encoded_o*
 3. Insight 3 - *Access_to_Resources_encoded_o*

Then, the selected input features were stored in *X* variable, while the target feature was stored in *Y* variable. The algorithm will use the features in *X* for training and predict the result for the target variable.

```

svm_model = SVC(kernel='rbf', C=1.0, gamma='scale', class_weight='balanced')
svm_model.fit(X_train, y_train)

y_pred = svm_model.predict(X_test)

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\n\nF1 Score:", round(f1_score(y_test, y_pred, average='weighted') * 100, 2), "%")
print("\nPrecision Score:", round(precision_score(y_test, y_pred, average='weighted') * 100, 2), "%")
print("\nRecall Score:", round(recall_score(y_test, y_pred, average='weighted') * 100, 2), "%")

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

print("\nAccuracy Score:", round(accuracy_score(y_test, y_pred) * 100, 2), "%")

```

Figure 4.8 Create and train SVM Model

The SVM model was configured using the Radial Basis Function (RBF) kernel to allow non-linear classification boundaries. The regularization parameter C was set to 1.0 to control the trade off between achieving low training error and a low testing error. The γ parameter was set to 'scale', which is the default setting in scikit-learn that adapts based on the number of features. Additionally, $class_weight = 'balanced'$ was applied to address the class imbalance by automatically adjusting the weights inversely proportional to class frequencies in the input data.

After training the model on the standardized training set using *fit()*, predictions were made based on the test set using *predict()* and were stored in the *y_pred* variable. The model was evaluated using various metrics such as confusion matrix, F1 score, precision, recall and accuracy. A classification report was also generated to provide a more detailed outcome on how well the model performed on each insight.

Classification Report:					
		precision	recall	f1-score	support
	0	0.30	0.54	0.39	13
	1	0.94	0.86	0.90	947
	2	0.70	0.88	0.78	307
	3	0.00	0.00	0.00	5
	4	0.00	0.00	0.00	4
	accuracy			0.86	1276
	macro avg	0.39	0.46	0.41	1276
	weighted avg	0.87	0.86	0.86	1276
Accuracy Score: 85.5 %					

Figure 4.9 Classification Report for Insight 1

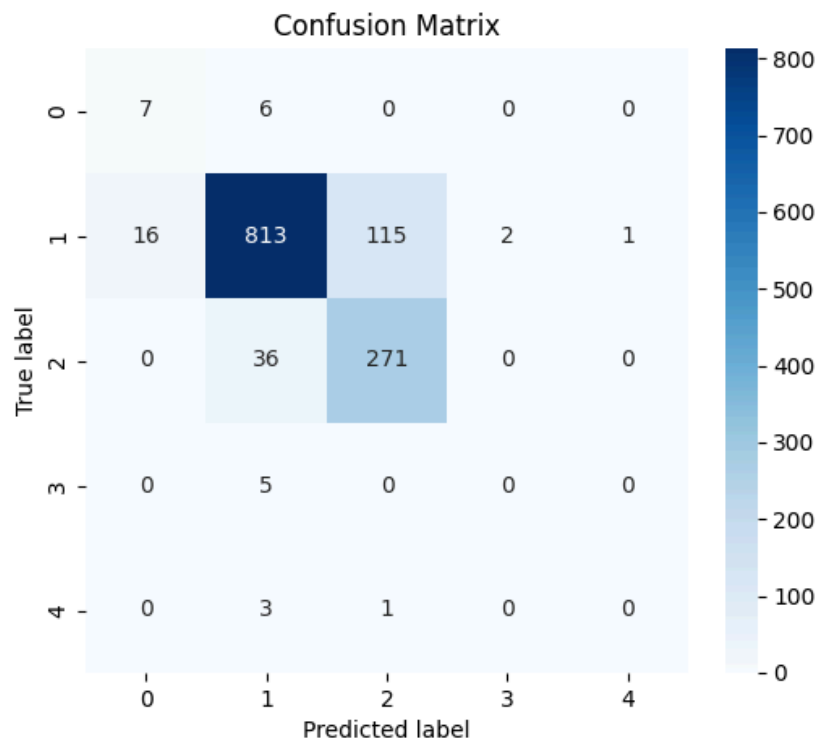


Figure 4.10 Confusion Matrix of SVM model of Insight 1

4.3 Deep Learning

This section delves into the application of Deep Learning, for predicting student outcomes within the dataset. As traditional machine learning algorithms may struggle with highly complex and non-linear data relationships, Deep Learning models, with their multi-layered architectures, offer a robust alternative for uncovering intricate patterns and enhancing predictive accuracy in such scenarios.

```
class DeepLearning(nn.Module):
    Windsurf: Refactor | Explain | Generate Docstring | X
    def __init__(self, input_features, hidden_features, output_features, dropout=0.3):
        super(DeepLearning, self).__init__()

        self.layer_stack = nn.Sequential(
            nn.Linear(input_features, hidden_features),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden_features, hidden_features // 2),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden_features // 2, output_features)
        )

    Windsurf: Refactor | Explain | Generate Docstring | X
    def forward(self, x):
        return self.layer_stack(x)
```

Figure 4.11 Deep Learning Class

This figure above shows a custom deep learning model defined using PyTorch's `nn.Module`. The architecture consists of a sequential stack of linear layers with non linear activation functions, Rectified Linear Units, ReLU, and Dropout layers. Combining the elements said earlier, the input layer takes the input features of `X.shape()` which is the number of features in our preprocessed dataset. Then two hidden layers are used to transform the input which are set to change from 128 on the first layer and 64 on the second layer. Then it is passed to ReLU to introduce the model to the non linearity of the data. After that, the data is passed onto the Dropout Layer that acts as a regularization that randomly sets 30% of the inputs from the previous layer to zero during training to prevent overfitting by forcing the network to learn more instead of blatantly memorizing the datasets. Lastly, the final linear layer maps the output of its

input that corresponds to the number of classes in target label Y. The raw output of logits are then created which will be passed onto the next phase in the training process.

```
for epoch in range(EPOCHS):

    ## Training phase
    model.train()
    # 1. Forward pass
    y_pred = model(X_train)
    # 2. Calculate loss
    loss = loss_fn(y_pred, y_train)
    # 3. Zero grad optimizer
    optimizer.zero_grad()
    # 4. Loss backward
    loss.backward()
    # 5. Step the optimizer
    optimizer.step()
    ## Testing phase
    model.eval()

    # 1. Forward pass
    with torch.inference_mode():
        test_pred = model(X_test)
        # 2. Calculate the loss
        test_loss = loss_fn(test_pred, y_test)

    if epoch % 10 == 0 or epoch == (EPOCHS - 1):
        print(f"Epoch: {epoch} | Train loss: {loss} | Test loss: {test_loss}")

    train_losses.append(loss.item())
    test_losses.append(test_loss.item())

return train_losses, test_losses
```

Figure 4.12 Training and Testing Model

Based on the figure of the function above, the model is trained over a predefined number of EPOCHS which is 250, in each epoch, it contains both training phase and testing phase. Inside the training phase, the input data went through forward passing (DeepLearning Class) to produce logits. Then, loss is calculated, which is the discrepancy between the logits and the true labels

value, `y_train`. After that, the gradients from the previous iteration are cleared to prevent accumulation using `optimizer.zero_grad()`. After the gradient is cleared, compute the new value of gradient with the modified value of each parameter in the model each time it iterates. The last process in the loop involved updating the model parameters based on the compute gradient and the learning rate(how fast the model learns) to make the model having less and less loss.

During the **testing phase**, the model's performance was assessed on the `X_test` data (without gradient computations for efficiency), providing an unbiased gauge of its generalization capabilities. Both training and testing losses were meticulously recorded to track the model's learning trajectory and identify any signs of overfitting or underfitting.

After training and testing, the model's performance was thoroughly evaluated using the dedicated `model_evaluation` function. We generated predictions on the unseen test data and then calculated key classification metrics: **accuracy**, **precision**, **recall**, and **F1-score**. These metrics provide a comprehensive understanding of how well our model performed, especially with the use of a weighted average to account for any class imbalances. To visually assess the training progress and identify issues like **overfitting**, we plotted the **training loss against the testing loss** across all epochs.

Finally, a **confusion matrix** was created as a heatmap. This visual tool offers a detailed breakdown of correct versus incorrect predictions for each class, helping us pinpoint where the model performed well and where it struggled. As an example, The confusion matrix is displayed in the figure below for Exam Score insight:

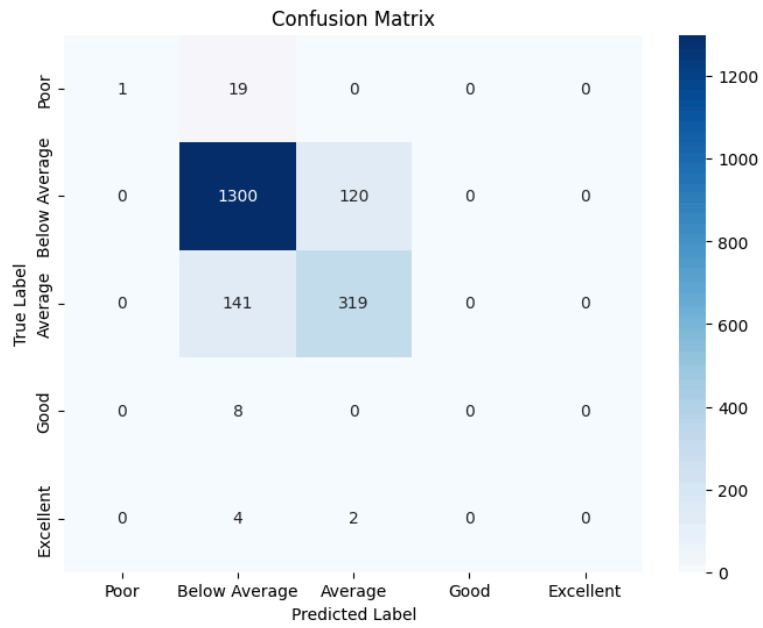


Figure 4.13 Confusion Matrix for Insight 1

5.0 DISCUSSION

5.1 Evaluation for Each Insight

Algorithms	Metric (%)	Insight 1	Insight 2	Insight 3
K-Nearest Neighbours	Accuracy	80.80	43.50	44.51
	F1 Score	78.69	39.11	40.48
	Precision	80.80	43.50	44.51
	Recall	78.09	40.29	41.14
Support Vector Machine (SVM)	Accuracy	85.50	32.52	38.17
	F1 Score	85.86	33.64	38.94
	Precision	87.07	36.78	42.47
	Recall	85.50	32.52	38.17
Deep Learning	Accuracy	84.59	48.90	49.97
	F1 Score	83.79	38.47	41.15
	Precision	83.94	41.40	42.32
	Recall	84.59	48.90	49.97

5.2 Observations

For insight 1, where the models predict student performance through Exam_Score_binned showing a significant class imbalance where the dataset mostly contain data towards the “Below Average” class which accounts to 74% and “Average” class which accounts to 24% while the data while remaining classes occupies only 2% of the entirety of the data. This causes the model to overwhelmingly predict only those two classes while ignoring other classes. This is common within our models since they were designed to optimize overall accuracy instead of balanced performance. This means the models will try to minimize loss by choosing the majority classes which are “Below Average” and “Average” even to the point of ignoring minor classes. Thus, the accuracy of predictions for all three models will always be good even though there is new data that do not fit the requirements to be included in the “Below Average” or “Average” classes, the models will be prone to predict that data as part of those classes nevertheless.

For insights 2 and 3, the common problem occurs between both of them due to their weak correlation among other features of the dataset. Most of the features show correlation values close to zero, which add noise instead of providing any useful values . Including such factors may decrease the model's accuracy by making it difficult to distinguish between relevant and irrelevant information. The features correlation for both insights are shown in figure below:

Correlation with target variable:		Correlation with target variable:	
Motivation_Level_encoded_o	1.000000	Access_to_Resources_encoded_o	1.000000
Parental_Education_Level_encoded_o	0.022089	Physical_Activity	0.038596
Hours_Studied	0.019137	Physical_Activity_binned	0.037906
Teacher_Quality_encoded_o	0.012535	School_Type_encoded_n	0.032939
Attendance	0.011524	Distance_from_Home_encoded_o	0.021567
Hours_Studied_binned	0.008960	Family_Income_encoded_o	0.012809
Distance_from_Home_encoded_o	0.006936	Tutoring_Sessions	0.011731
Peer_Influence_encoded_o	0.004345	Parental_Involvement_encoded_o	0.009756
Attendance_binned	0.003546	Peer_Influence_encoded_o	0.009121
Sleep_Hours	0.002885	Attendance_binned	0.008868
Gender_encoded_n	0.000599	Teacher_Quality_encoded_o	0.007870
Internet_Access_encoded_n	0.000574	Tutoring_Sessions_binned	0.007618
Physical_Activity_binned	-0.001650	Gender_encoded_n	0.007088
Physical_Activity	-0.001658	Learning_Disabilities_encoded_n	0.006179
Sleep_Hours_binned	-0.001878	Internet_Access_encoded_n	0.004008
School_Type_encoded_n	-0.005003	Attendance	0.003485
Tutoring_Sessions_binned	-0.005311	Hours_Studied_binned	-0.000482
Extracurricular_Activities_encoded_n	-0.006446	Previous_Scores	-0.003683
Family_Income_encoded_o	-0.008073	Sleep_Hours	-0.004019
Access_to_Resources_encoded_o	-0.009307	Sleep_Hours_binned	-0.006336
Parental_Involvement_encoded_o	-0.010856	Extracurricular_Activities_encoded_n	-0.006487
Tutoring_Sessions	-0.011804	Parental_Education_Level_encoded_o	-0.007518
Exam_Score	-0.017328	Motivation_Level_encoded_o	-0.009307
Learning_Disabilities_encoded_n	-0.017858	Previous_Scores_binned	-0.009354
Previous_Scores	-0.021640	Hours_Studied	-0.010011
Exam_Score_binned	-0.022394	Exam_Score	-0.090258
Previous_Scores_binned	-0.022879	Exam_Score_binned	-0.091435

Figure 5.1 Correlation Values Between Target Variables and Other Features for Insight 2 and Insight 3

6.0 CONCLUSION

In conclusion, the different performance results happen because each algorithm works in different ways and the data relationships vary in complexity. SVM performs very well on Insight 1 because it can draw clear lines to separate different classes, which works well when attendance and exam scores have a straightforward relationship. However, SVM has trouble with Insights 2 and 3 because motivation levels and resource access have more complicated connections that cannot be easily separated by simple lines. Deep Learning works better for these complex tasks because its multiple layers can learn complicated patterns that other algorithms cannot handle. The poor results for all models on Insights 2 and 3 show that these relationships are very difficult to predict, suggesting that the current data may not have enough information to accurately predict motivation levels and resource access.

7.0 REFERENCES

1. Fhea, D. C. P. (2024, June 25). *Correlation in supervised, unsupervised, and reinforcement learning*.
<https://www.linkedin.com/pulse/correlation-supervised-unsupervised-reinforcement-dilek-celik-phd-nabcf>
2. GeeksforGeeks. (2024, September 13). *What is Imbalanced Dataset*. GeeksforGeeks.
<https://www.geeksforgeeks.org/what-is-imbalanced-dataset/#why-imbalanced-datasets-are-a-problem->
3. *Datasets: Imbalanced datasets*. (n.d.). Google for Developers.
https://developers.google.com/machine-learning/crash-course/overfitting/imbalanced-datasets?utm_source=chatgpt.com
4. *mrdbourne/pytorch-deep-learning: Materials for the Learn PyTorch for Deep Learning: Zero to Mastery course*. (n.d.). <https://github.com/mrdbourne/pytorch-deep-learning>